ISIS-1611 Taller 2

Home Content Contact

ISIS-1611

TALLER 2

CSP & ADVERSARIAL SEARCH

Start

RECURSION

1. Define base case
2. Define recursive case
3. Ensure recursion terminates
4. Combine the solutions

Recursion Visualization

```
def sum(n):
    # base condition
    if n == 1:
        return 1

    return n + sum(n - 1)
```

sum(4)=?

ESTRUCTURA

- Drones/
  - main.py
  - algorithms/
  - layouts/
  - view/
  - world/

CSP

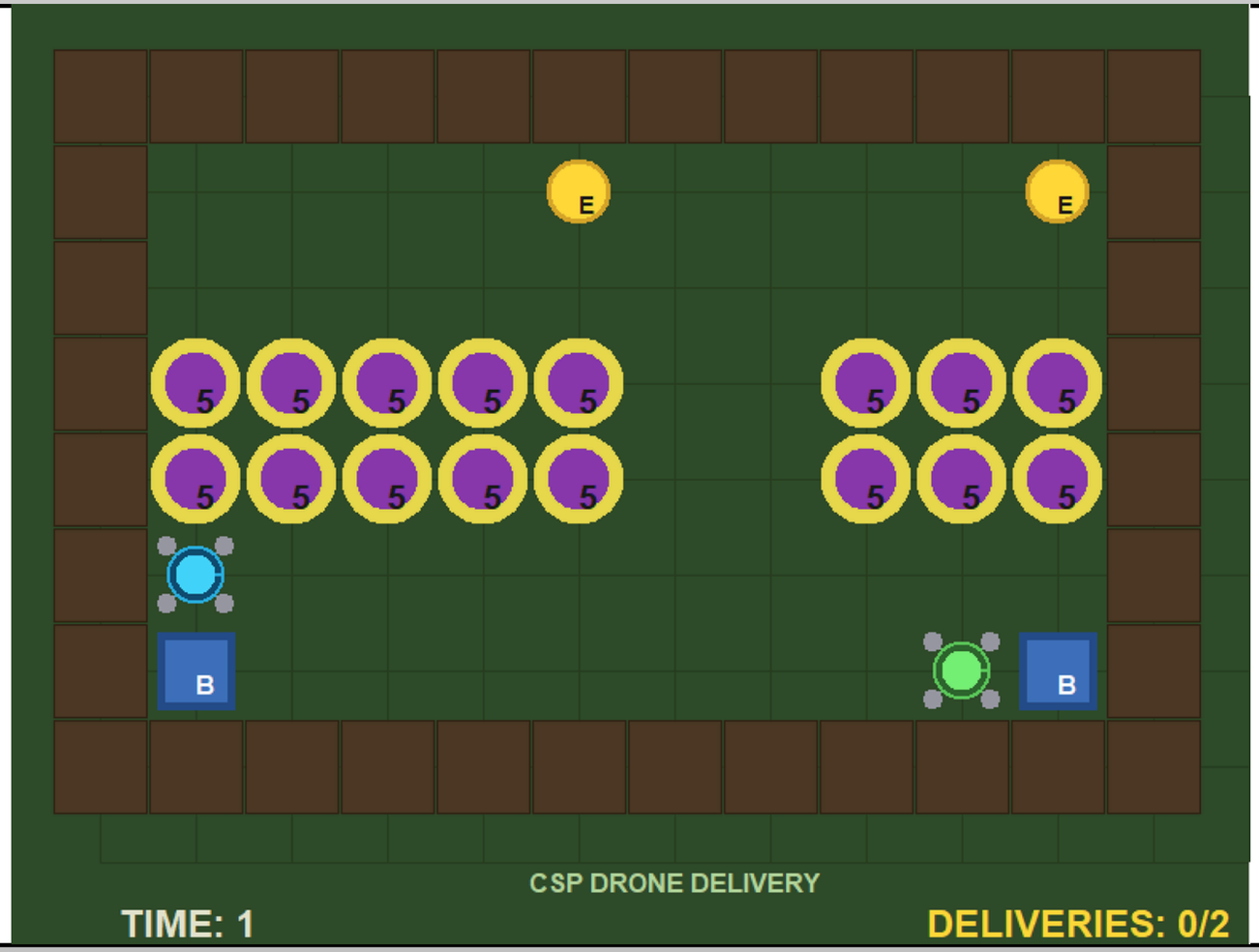
- Variables
- Domain
- Constraints

Drones/algorithms/csp.py

- backtracking_search
  - Basic backtracking
- backtracking_fc
  - with forward checking
- backtracking_ac3
  - with AC-3 arc consistency
- backtracking_mrv_lcv **(Bono 5%)**
  - with both MRV and LCV heuristics

ISIS-1611 Taller 2

Home **Content** Contact



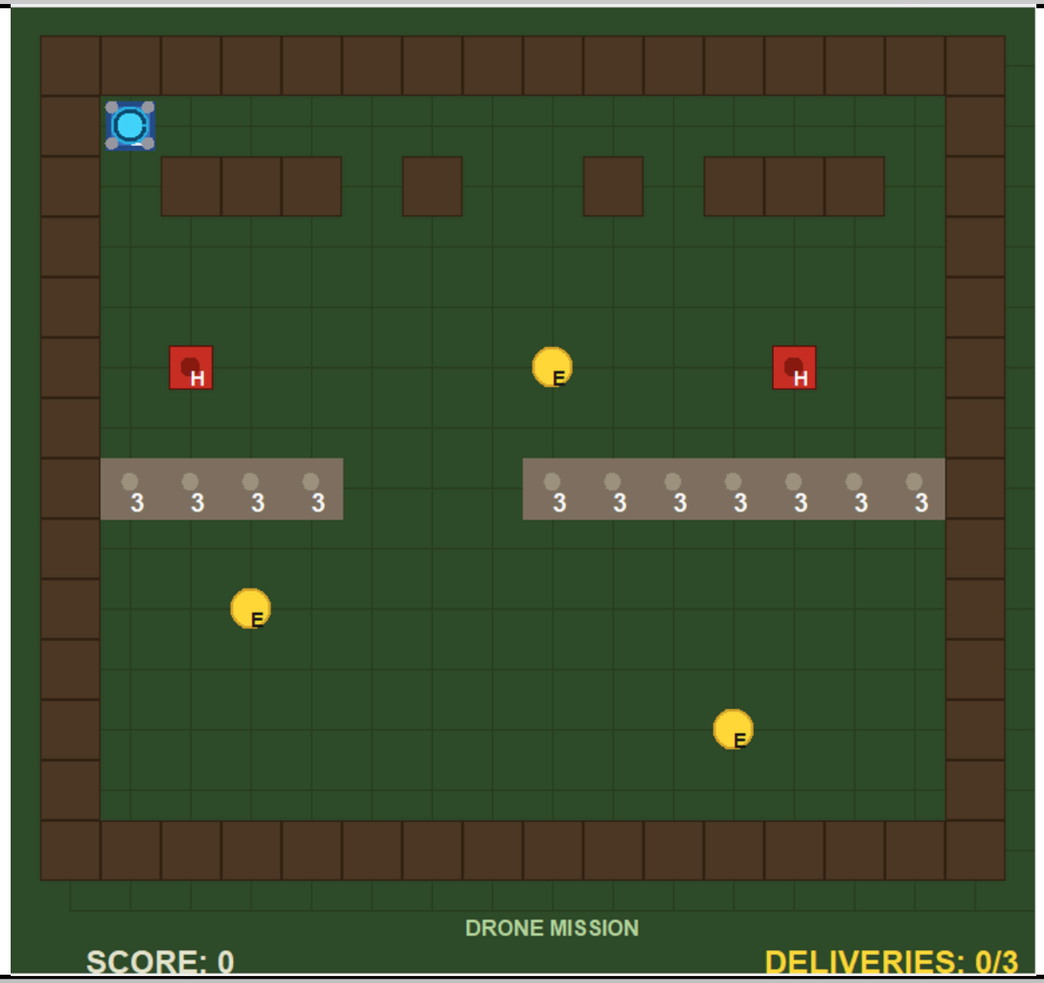
ADVERSARIAL

Drones/algorithms/adversarial.py

- RandomAgent
  - Random legal action
- MinimaxAgent
  - Best action with minimax
- AlphaBetaAgent **(Bono 5%)**
  - Optimized with alpha-beta
- ExpectimaxAgent
  - Uses chance nodes to better represent problem
  - Run multiple games with *-n*

ISIS-1611 Taller 2

Home **Content** Contact



EVALUATION

Drones/algorithms/evaluations.py

- Assigns value to non-terminal states

Drones/algorithms/utils.py

- bfs_distance
- dijkstra
- manhattan_distance

Revisen docstring para entender funciones